Collaborative Document RSS day 4

2026-09-15-research-software-support-training

Welcome to The Workshop Collaborative Document.

This Document is synchronized as you type, so that everyone viewing this page sees the same text. This allows you to collaborate seamlessly on documents.

This is the Document for today: https://edu.nl/mwc48

Collaborative Document day 1: https://edu.nl/nmwaj

Collaborative Document day 2: https://edu.nl/jnqpv

Collaborative Document day 3: https://edu.nl/g6eqm

Collaborative Document day 4: https://edu.nl/mwc48

Collaborative Document day 5: https://edu.nl/6jm6x

Code of Conduct

Participants are expected to follow these guidelines:

Use welcoming and inclusive language.
Be respectful of different viewpoints and experiences.
Gracefully accept constructive criticism.
Focus on what is best for the community.
Show courtesy and respect towards other community members.
If you feel that the code of conduct is breached, please talk to one of the instructors (if the complaint is for one of the participants) or send an email to training@esciencecenter.nl (if the complaint is for one of the instructors).

Certificate of attendance

If you attend the full workshop we will send you a certificate of attendance.
We will delete all personal identifyable information 8 months after the training.

⚖ License

All content is publicly available under the Creative Commons Attribution License: creativecommons.org/licenses/by/4.0/.

Getting help

To ask a question, raise your hand in zoom. Click on the icon labeled “Reactions” in the toolbar on the bottom center of your screen,
then click the button ‘Raise Hand ’. For urgent questions, just unmute and speak up!

You can also ask questions or type ‘I need help’ in the chat window and helpers will try to help you.
Please note it is not necessary to monitor the chat - the helpers will make sure that relevant questions are addressed in a plenary way.
(By the way, off-topic questions will still be answered in the chat)

Workshop website & Lesson materials

RSST Sept-Oct 2026
https://esciencecenter-digital-skills.github.io/2026-09-15-ds-rss-masterclass/

RSST lesson material
https://esciencecenter-digital-skills.github.io/research-software-support/

Instructors

Sander van Rijn
Jaro Camphuijsen
Ewan Cahen
Ole Mussmann

Helpers

Fenne Riemslagh
Eva Lekkerkerker

Agenda

Time Topic
09:00 Welcome and recap
09:15 Distributing Software
10:15 Break
10:30 Software Publications
11:30 Break
11:45 Software Licenses
12:45 Wrap up
13:00 End of day 4

Questions from last week

Software Testing

Q: Is there a test that is better handled manually rather than automated?
A: User experience-type testing is better tested manually, but even then there is a lot of things that you can test automated; does the page respond on time? Smoke-tests would also be something to do manually regularly - it's not really aimed at specific parts. A lot of system-tests also double as smoke-test. Changing visual changes is also hard to test in an automated manner. However, always test automatically if you can. Everything you have to do manually takes longer, and is more mistake-prone.

Q: is evaluating a software compatability with operating system also a type of regression test?
A: Reserve the term "regresson testing" for changes that are happening in your own code. I think it falls under system test where you're checking how all components work together. Testing software compatability with operating system can definitely be part of testing. All the testing types are not necessarily mutually exclusive. If you run a unit test regularly, it can be a form of regression test.

Q: When providing support on testing (for instance emphasizing to researchers that it is an important aspect of software development when reviewing SMPs) I was wondering if there are useful resources I could refer (perhaps rather unexperienced coders) to?
A:

- For Python: Getting Started with Pytest (https://docs.pytest.org/en/stable/getting-started.html)
- For R: Testing Basics using Testthat (https://r-pkgs.org/testing-basics.html)
- We have more links on the content website (https://esciencecenter-digital-skills.github.io/research-software-support/modules/testing/reading)

Citation

Q: Is there a fixed list of keywords one can use for the CITATION.cff file?
A: For domain specific keywords, you could use the New ERC panel structure in 2021 and 2022 (https://erc.europa.eu/news/new-erc-panel-structure-2021-and-2022) for example (link to the PDF (https://erc.europa.eu/sites/default/files/document/file/ERC_Panel_structure_2021_2022.pdf)), which is used at the RSD to categorise projects. Another option is the NWO research fields (https://www.nwo.nl/en/nwo-research-fields).

Note that a large part of research software is domain agnostic, so the lists above are not applicable. In this case, think about technical keywords to describe the software, i.e. `GPU programming` or `distributed computing`.

Collaborative Notes

Software Distribution

Why distribute?

- For reproducibility/transparency of research
- For deveopers that want to reuse parts of your software/methods
- For users

Functions or Procedures

- Ordered list of instructions
- Like a recipe
- Can refer to other recipes
- Closed box

Scripts and Notebooks

- Single file with function(s)

- Sit at the top of a pyramid (on top of the libraries you've loaded)
- Use libraries
- Run interactively
- User handles errors

Libraries

- Libraries are organized around a certain topic of functions.
- Ready-made functionality. It has functions you don't have to program yourself anymore, because you can run them from the library.
- Have an interface
- Handle errors themselves.

programs
A program is bigger than a script, but it's a bit fuzzy where a program begins and a script ends. It's an end product with a user interface.

Packages

- Archives with a collection of software
- Intended for distribution
- Usually holds a library or a program
- Has metadata
- Listed in a package index - like Pypy in Python or CRAN in R

Images and containers

- A file containing a collection of software
- Everything including the OS
- Cannot be combined
- Can be run like a program, as a container

Cannot easily be combined or called from the script. Suited for programs and not as much for libraries.

Distributing scripts/notebooks

- Make script/notebook available for download
- Create an environment file (requirements.txt, environment.yml)
- User uses a package manager to create environment
- User runs the script/notebook
- Optional: use an image

Distributing libraries

- Make a package containing the library
    - Most modern systems (R, Python, HPC...) have a packaging system available
- Add metadata with dependencies
- Upload the package to a repository

Distributing programs

- Put it in an app store! packaging system for applications like AppStore or GooglePlay
- As an installer? - warning that some applications might contain virus, but if it comes

from a verified distributor typically a sign of trustworthiness
- As a package
- As an image - have to update your images quite regularly, it is a bit more work

Updating software

- Scripts are low maintenance. it's easy to approach and typical user can work around errors and update the script.
- For libraries and programs maintenance is required. People expect different user experience with time as well.
- This is hard to do in project-based science. You can publish on GitHub for example and archive it. This would signal to users that it's still available but no longer actively maintained.

Versioning

- Typically Semantic Versioning for libraries (Major.Minor.Patch)

Publishing Software

Why publish?

- Major scientific considerations with citeability and reproducibility
- FAIRness

Modern publishing pipeline

- Have a software repository
- Versioning your software
- Get a persistent identifier
- Registries
- Publish software paper

Software repository
Put all source files in a public version-controlled repository e.g. GitHub, GitLab

Software is immediately Findable
Naturally contains a log of all development history
Include a software license, and a citation.cff file

Which software repositories are there?

- BitBucket
- Launchpad
- Github
- Codeberg
- Gitlab
- Codegiant
- Azure Repos
- ....

Tag/semantic versioning
It is important to communicate the actual version of the software used for a particular piece of research

If your software is used as a dependency of another software/workflow then specific versions can be referenced

Ideally, use "semantic versioning" in the form MAJOR.MINOR.PATCH (e.g. v2.1.5)

Get a Persistent Identifier
Specific versions of the software should be published to an archiving service that provides persistent identifiers.

For example, Zenodo is a long term archiving service for scientific data and software. Zenodo will assign a Digital Object Identifyer (DOI) to the published data, making it citable.

Registries
Add a link to your software on a relevant public registry

For example, the eScience Center maintains the Research Software Directory
Can also be domain specific software registries
Or even list it on awesomelist
This aids greatly in Findability by relevant researchers

The Research Software Directory (RSD)

The Research Software Directory is an online service designed to collect and present “related research information” in a structured way. Our goals:

- Enable RSEs to show the impact their software has in research and society
- Help researchers to find research software they need in order to perform their research
- Allow organisations to collect information about the research software output they produce
- Encourage citation of research software in other research outputs

Free and easy to use
Sign-in via SURFconext, ORCID, GitHub, LinkedIn
We automatically harvest data where possible:

- Contributors via ORCID / DOI+CFF
- Organisations via ROR
- Keywords & licences from DOI
- Releases & mentions via DOI
- Descriptions via GitHub/GitLab
- Citations via reference papers
- Downloads via package manager

Software Paper
Publish a journal article about the software specifically

Journal of Research Software (JORS)
Journal of Open Source Software (JOSS)
SoftwareX
CRAN
Some journals perform peer review of the software in question

Automate as many steps as possible!
For example, you can automate archiving on Zenodo by linking it to your GitHub account

See instructions in the "Automating software publication" tab of this story

Software Licensing

Common questions

- Can I use this software I found online?
- Can I share my software with others?
- Can I contribute to someone else's software?

Copyright

- Berne convention on copyright (late 1800s)
- National and European law on copyright
- Owned by author or employer?
- Can be licensed

Using other's software

- Two ways of using software
    - Downloading a script and editing it
    - Downloading a library and building on it
- You need a license
    - Just because its on the internet doesn't mean that you're free to modify it
- Licenses sometimes implicit or home-made
    - Home-made: still (supra)national licenses overrule
    - Sometimes ambiguous categories between industry and academia
- Open Soure is better and safer
    - Easiest to use conventions in Open Source licensing
- Different Open Source licenses available

Why share software?

- Reproducibility
- Increasing impact
- Academic freedom of employees
- Saving by collaborating
- Public funding

If you produce software within the scope of your employment by an academic institute, your employer is officially the owner of that software. By publishing your software with an open license, you're also legally allowed to continue modifying it in the future when you move to a different employer.

How to share software

- Put it online
- Add a license
    - Who owns the software?

    - Dependencies
    - Kind of licenses

Who owns a work?

- Typically the employer they worked for.
    - Not always apparent at first glance.
- Consortium agreements.
    - If someone worked in a temp. project under a consortium agreement it can be difficult to deduce who owns the software.
- Funding constraints
    - If you have commercial funding maybe they claim ownership. eScience Center for example has a contractual agreement with our funders to produce open source.

Kinds of licenses

Some differences within Free and Open Source licenses

Permissive: BSD, MIT, Apache
Copyleft - Strong: GPL, AGPL
Copyleft - Weak: LGPL, MPL
Proprietary - Research only; no copying, no modification.

If you copy or modify a package/container, parts of the libraries might have different license restrictions than others. For example: you might have to republish it with the source code in a GPL licensed library - only applied on that specific library.

Choosing a license

- Use a standard Open Source (software) license
- Check your policy (if applicable)
- Use a tool: https://choosealicense.com/

Contributing to software

- Under the license
- Contributor License Agreements
    - Who are you dealing with?
    - Transfer or license?

Sometimes you have a project that requires to sign a contributor license first; and if you're contributing as part of a work project, your boss will have to sign. Question who you are dealing with: Google or Apache? Is it commercial or public?

Important: are you transferring your copyright - losing ownership? Or do you actually contribute - that all contributors have properly licensed (Apache). The Apache foundation asks you to transfer your copyright ownership so that they're able to rerelease under new copyright, but they are dedicated to open source so it's OK. However, with Google I wouldn't transfer my copyright.

Excercise

Distributing Software

https://esciencecenter-digital-skills.github.io/research-software-support/modules/distributing/exercise-distributing

Room 1

Room 2

- Case 1:
  Define target audience = researchers, then app store would not be really the way to go, despite the mentioning of application. Distribute it as a package. Considered maintenace.

Room 3

Case 1:

We would ask following questions to figure out if it is really a whole application or more a small group of scripts.

1. If it is indeed an application, so we advise to distribute it as a **program**.
2. If it is a small collection of scripts, to help researchers to build upon it, we would advise share it as a **package**.

Room 4

Room 5

Publishing Software

Register your software in the Research Software Directory (15 mins)

https://esciencecenter-digital-skills.github.io/research-software-support/modules/publication/register_software_rsd_exercise/

Use the RSD Playground: https://research-software.dev/

Finding Your Registry

https://esciencecenter-digital-skills.github.io/research-software-support/modules/publication/ex_registries

Awesome List of Registries: https://github.com/NLeSC/awesome-research-software-registries

Room 1

Room 2

Room 3

Room 4

Room 5

Software Licensing

Exercise 1: Find the license for a software package (10 minutes)

https://esciencecenter-digital-skills.github.io/research-software-support/modules/licenses/ex_finding_the_license

Imagine that a researcher comes to you and asks whether they can use a particular software package that they found online. One aspect to consider would be its license, if there is one.

Pick a software package from the following list, and try to find out whether it has a license and if so which. The idea is to cover the list together, so let's not all start at the top!

Kernel Tuner - Apache 2.0

nodal-dg - MIT

OpenFOAM - GNU General Public License version 3

MATLAB - Closed Source proprietary license
- Trying to find anymore info on this because a researcher asked me about it recently. Only info I can find is https://nl.mathworks.com/help/matlab/matlab_env/manage-your-licenses.html

COPASI: Artistic license 2.0: allows non-commercial and commercial use free of charge

Glotaran: GPL v2 or later (copy or distribute free with same copyright and disclaimer, modifications allowed but shared under the same licence or later version of it)

LinkAhead - GNU AGPLv3

A few hints:

The researcher only gave you the name. You'll have to find the code on the Internet first. Licenses are not always advertised on a code's website. Try looking somewhere more technical if you can't find it in the documentation.

Discussion:

- What MATLAB calls a license isn't a copyright license. It doens't give you any permission. What those licenses do is list all the things you're NOT allowed to do. They are contracts that serve to enforce limitations on top of Copyright law. For example; you have to pay more if you have more users. It falls under Contract law.
-

:car: Parking lot

Questions from this week

Q: I often use the terms library and package interchangeably, what would you say is the difference?
A: A package can contain libraries, and the library is usually packaged into the software

package. The terms are often used interchangeably. Packaging is a technique for collections of software: libraries are quite often the packages that people will use, but it might also contain a software.

Q: How is the RSD different from Software Heritage?
A:

- Software Heritage is only for the *source code* of software, while Zenodo is also for *data, posters, slides and other artefacts*.
- Software Heritage is about *all* software, while Zenodo is about *open science*.
- Software Heritage only archives the source code, while Zenodo also allows citation, has communities, has more extensive search features.

Have a look at the about page of Zenodo (https://about.zenodo.org/) and the mission statement of Software Heritage (https://www.softwareheritage.org/mission/). Since recently, Zenodo and Software Heritage integrate with each other (https://blog.zenodo.org/2024/10/21/2024-10-21-swh/).

Q: Why would you choose for Zenodo, comparative to other options like 4TU.ResearchData, where you also get a DOI.
A: Zenodo is for archiving long-term. 4TU seems more like a GitHub alternative. It doesn't give the scientific context of the software, where the RSD harvests that information on a landing page for your software.

Q: Does GitLab have Zenodo integration just like GitHub has?
A: No, this, this feature request from 2018 (https://github.com/zenodo/zenodo-rdm/issues/1203) is still open. Have a look at this unofficial script (https://gitlab.com/deploy2zenodo/deploy2zenodo).

Q: If I edit the MATLAB Toolbox enough, can I contribute my changes or license my changes?
A: If it's a library in MATLAB Toolbox, then that library might have a separate license that allows modification. That is something you will have to check. If a separate license doesn't exist, MATLAB restricts you. MATLAB has file-exchange where people can make Toolboxes and publish them. MATLAB automatically assign DSB open license, but you can probably also choose a different open source license.

Q: What about software that you create outside op work hours.
A: If you make software outside of workhours on your worklaptop, ownership can be claimed for work. If you make software intended for work related activities on your personal laptop, that can also be regarded as owned by your employer.

Q: If someone uses AI to rewrite your software under GPL?
A: it's unclear what judicial outcomes will be of copyright cases that are inspected right now. It's a grey area how a derivative work falls under software copyright. You do need copyright permission to train model on data - also being litigated right now. If the structure and story of the code is roughly the same it's still copyright infringement.

Q: in what category falls the creative common license? and what is the difference between CC and these licenses?
A: The licenses listed are software licenses - they are older than the Creative Commons licenses. CC were inspired by these licenses. Not all are akin to open source - like no-derivative licenses. The GPL makes a clear distinction between source code and library - but CC' licenses

do not use such distinction. Software licenses offer more specificity. If you publish software - use a specific software license. If you publish your presentation about the software, or a blog you wrote, or learning materials - use Creative Commons.

Q: Maybe too specific but: How does this work with dual licensing? If I remember correctly I have seen things from eScience center both under a GPL and apache license simultaniously
A: Dual licensing can happen if you have software that has multiple sources. If you have your pyramid of libraries under your script/notebook, you have to follow the licenses of all those libraries. If the licenses are different it is still possible to distribute the whole pyramid, but you need to adhere to all those licenses simultaneously, and use the right license for the appropriate brick of code. Some licenses are more compatible than others - like GPL & MIT. If you have proprietary licenses then it may become a problem - perhaps the proprietary license doesn't allow sharing source code, while GPL forces you to share source code. That becomes an issue.

Q: We often share mainly data, but also with developed software within that same collection. And then the data considerations usually prevail. What are your thoughts on this?
A: If your software is under a Creative Commons license, it depends which one it is. It will be difficult to combine that software with other software under an open source license. If it's a non-derivative CC license that is not compatible with open source. If its CC-0 that is basically compatible with open source software licenses. If the software is a finished application to work with that data - then maybe its not an issue in practice. Still recommend to split it off - and use a separate license for open source software license.

Q: What if you (want to) change and/or add a license at a later date? Can you do that and does it apply retroactively?
A: Yes, you can change a license. But you cannot withdraw an open source license. If people downloaded it under an open source license, you cannot retroactively restrict those users access to the software. Once someone has that license, they get to keep it. If you want to change the license, you need permission from all the copyright owners. If you're the main person - that's not so difficult. But if it's Linux for example - they have thousands of copyright holders - you would require permission from everyone. Typically it's resolved by making a new release, and older releases would remain under their old license. Sometimes copyright holders are not available to reach - you would have to redevelop their parts of the software.

:book: Homework / Resources

Distributing Software

- (Optional)(Docker) More about containers & container images:
    - https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/
- Distributing Software Reading
    - https://esciencecenter-digital-skills.github.io/research-software-support/modules/distributing/distributing
- The "Why" of VM's, Containers and Virtual Environments
    - https://esciencecenter-digital-skills.github.io/research-software-

support/modules/distributing/further-reading

Software Publication

- Further reading: https://esciencecenter-digital-skills.github.io/research-software-support/modules/publication/reading

Software Licenses

- Choosing a license tool: https://esciencecenter-digital-skills.github.io/research-software-support/modules/licenses/ex_choose_a_license/